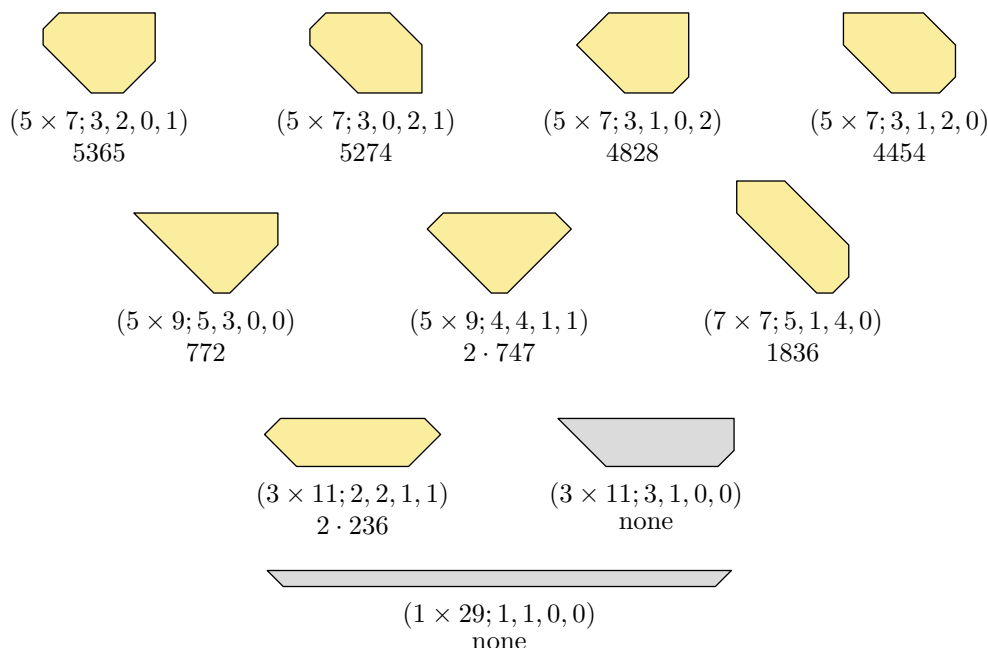


September 6, 2026 at 11:44

1. Introduction. Exercise 7.2.2.1–320 asks two questions. First, how do you enumerate all of the convex N -aboloes? Second, how many of the convex 56-aboloes can the fourteen tetraboloes pack?

Answer 320 answers both, and it hands out a lot of numbers on the way: a convex polyabolo is a rectangle with its four corners cut off, there are 63 convex 56-aboloes, exactly five tetraboloes are odd in a sense due to O’Beirne, only ten of the 63 survive the parity test that follows, two of those ten are impossible anyway, and the remaining eight have 1836, $2 \cdot 236$, 772, $2 \cdot 747$, 5365, 5274, 4828 and 4454 packings. This program works all of those numbers out from scratch.

Here are the ten shapes that pass the parity test. Eight of them can be packed, and the number of ways is printed underneath; the two greyed ones cannot, though only a search can say so.



```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strings"
    "time"
    cells "github.com/sjnam/dancing-cells"
)
<Types 3>
<Functions 4>
func main() {
    <Read the command line 2>
    <Do what the mode asks 41>
}
```

2. Four modes, and by default all four run in turn. *census* grows every polyabolo up to a given size and picks out the convex ones, which is the independent check that the six parameters of answer 320 really describe all of them; *pieces* checks the fourteen tetraboloes and O’Beirne’s parity; *convex* lists the convex 56-aboloes and applies the parity test to them; and *pack* counts the packings.

⟨ Read the command line 2 ⟩ ≡

```
mode := flag.String("mode", "all", "census, pieces, bound, convex, pack, or all")
size := flag.Int("N", 56, "the number of halvesquares")
upto := flag.Int("upto", 11, "how far the census grows")
flag.Parse()
```

This code is used in section 1.

3. The grid. A polyabolo is built from isosceles right triangles glued edge to edge, and the natural triangle is half of a unit cell of the square grid. The awkward part is that a cell can be cut along either diagonal, so it offers four possible halves and two of them may overlap. Answer 319 shows the way out: cut every cell at *both* diagonals, into four quarters, and note that each halfsquare is two adjacent quarters. On quarters the pieces are simply disjoint sets, which is exactly what an exact cover problem wants.

So there are two levels here. A quarter is **hs**, and a halfsquare is **half**.

⟨Types 3⟩ ≡

```
type hs struct { x, y, q int } // quarter q of cell (x, y): 0 south, 1 east,
// 2 north, 3 west
type half struct { x, y, t int } // the halfsquare owning quarters t and
// t − 1
type shape []hs
type piece []half
```

This code is also defined in section 23.

This code is used in section 1.

4. I number the halves so that *t* owns quarters *t* and *t* − 1 modulo 4: 0 is the lower left half, 1 the lower right, 2 the upper right, 3 the upper left. Halves 0 and 2 are the two sides of the “\” diagonal, halves 1 and 3 the two sides of the “/” diagonal; two halves of one cell fit together only when they are a pair of that kind, and the quarters say so by themselves.

⟨Functions 4⟩ ≡

```
func (h half) quarters() []hs {
    return []hs{ {h.x, h.y, h.t}, {h.x, h.y, (h.t + 3) % 4} }
}
```

This code is also defined in sections 5, 8, 10, 11, 13, 15, 16, 19, 20, 24, 25, 27, 29, 30, 33, 34, 35, 36, 37, 40.

This code is used in section 1.

5. The eight symmetries of the square grid act on these coordinates by short formulas, which is the whole reason for choosing them over the pixel coordinates of answer 319. A quarter turn counterclockwise about the origin sends cell (x, y) to $(-y - 1, x)$ and advances the quarter; a reflection in the vertical axis sends it to $(-x - 1, y)$ and swaps east with west, which on halves is the single bit $t \oplus 1$.

⟨Functions 4⟩ +≡

```

func transform(h hs, k int) hs {
  if k ≥ 4 {
    ⟨Reflect the quarter 6⟩
  }
  for i := 0; i < k % 4; i++ {
    ⟨Turn the quarter 7⟩
  }
  return h
}

func transformH(h half, k int) half {
  if k ≥ 4 {
    h = half{-h.x - 1, h.y, h.t ⊕ 1}
  }
  for i := 0; i < k % 4; i++ {
    h = half{-h.y - 1, h.x, (h.t + 1) % 4}
  }
  return h
}

```

6. ⟨Reflect the quarter 6⟩ ≡

```

q := h.q
if q % 2 ≡ 1 {
  q = (q + 2) % 4
}
h = hs{-h.x - 1, h.y, q}

```

This code is used in section 5.

7. ⟨Turn the quarter 7⟩ ≡

```

h = hs{-h.y - 1, h.x, (h.q + 1) % 4}

```

This code is used in section 5.

8. A polyabolo is a region of the plane, so I identify one by the set of quarters it covers: that makes the two ways of cutting a full cell the same shape, as they should be. The key of a shape is its quarters sorted after the shape has been slid down to the origin.

⟨ Functions 4 ⟩ +≡

```

func (s shape) norm() shape {
  t := append(shape{}, s...)
  ⟨Slide the shape to the origin 9⟩
  sort.Slice(t, func(i, j int) bool {
    if t[i].y ≠ t[j].y {
      return t[i].y < t[j].y
    }
    if t[i].x ≠ t[j].x {
      return t[i].x < t[j].x
    }
    return t[i].q < t[j].q
  })
  return t
}

func (s shape) key() string {
  var b strings.Builder
  for _, h := range s.norm() {
    fmt.Fprintf(&b, "%d,%d,%d;", h.x, h.y, h.q)
  }
  return b.String()
}

```

9. ⟨Slide the shape to the origin 9⟩ ≡

```

mx, my := t[0].x, t[0].y
for _, h := range t {
  if h.x < mx {
    mx = h.x
  }
  if h.y < my {
    my = h.y
  }
}
for i := range t {
  t[i].x -= mx
  t[i].y -= my
}

```

This code is used in section 8.

10. The eight images of a piece, deduplicated, are its orientations; the least of their keys is its canonical name. Reflections count, since the fourteen tetraboloes are the free ones.

⟨ Functions 4 ⟩ +≡

```

func (p piece) quarters() shape {
  var s shape
  for _, h := range p {
    q := h.quarters()
    s = append(s, q[0], q[1])
  }
  return s
}

func (p piece) key() string { return p.quarters().key() }

func (p piece) orients() []piece {
  seen := map[string]bool{ }
  var out []piece
  for k := 0; k < 8; k++ {
    t := make(piece, len(p))
    for i, h := range p {
      t[i] = transformH(h, k)
    }
    if s := t.key(); ¬seen[s] {
      seen[s] = true
      out = append(out, t)
    }
  }
  return out
}

func (p piece) canon() string {
  best := ""
  for _, t := range p.orientations() {
    if k := t.key(); best ≡ "" ∨ k < best {
      best = k
    }
  }
  return best
}

```

11. Growing polyaboloes. To check the characterization of answer 320 against something that knows nothing about it, I grow every polyabolo one triangle at a time and keep the ones that come out convex. The counts of polyaboloes themselves are a free check on the geometry: 1, 3, 4, 14, 30, 107, 318, 1116, 3743, 13240, 46476 for $N = 1$ to 11, and in particular the three diaboloes and the fourteen tetraboloes that the exercise names.

⟨ Functions 4 ⟩ +≡

```

func growH(upto int) [[]]piece {
  all := make([[]]piece, upto + 1)
  all[1] = []piece{half{0, 0, 0}}
  for n := 2; n ≤ upto; n++ {
    seen := map[string]bool{ }
    for p := range all[n - 1] {
      ⟨ Add one triangle to p in every way 12 ⟩
    }
  }
  return all
}

```

12. A halfsquare touches its partner across the diagonal, and across each of its two outer sides it touches either of the two halves of the next cell that own the facing quarter—five candidates, of which the ones that would overlap the piece are thrown away.

⟨ Add one triangle to *p* in every way 12 ⟩ ≡

```

used := map[hs]bool{ }
for h := range p {
  for q := range h.quarters() {
    used[q] = true
  }
}
for h := range p {
  for g := range neighbours(h) {
    q := g.quarters()
    if used[q[0]] ∨ used[q[1]] {
      continue
    }
    t := append(append(piece{ }, p...), g)
    if k := t.canon(); ¬seen[k] {
      seen[k] = true
      all[n] = append(all[n], t)
    }
  }
}

```

This code is used in section 11.

13. $\langle \text{Functions } 4 \rangle + \equiv$

```

func neighbours(h half) []half {
  out := []half{ {h.x, h.y, (h.t + 2) % 4} }
  for _, s := range []int { h.t, (h.t + 3) % 4 } {
    x, y := h.x, h.y
     $\langle \text{Step across side } s \text{ } 14 \rangle$ 
    u := (s + 2) % 4
    out = append(out, half{x, y, u}, half{x, y, (u + 1) % 4})
  }
  return out
}

```

14. $\langle \text{Step across side } s \text{ } 14 \rangle \equiv$

```

switch s {
case 0:
  y --
case 1:
  x ++
case 2:
  y ++
default:
  x --
}

```

This code is used in section 13.

15. Convexity. A polyabolo is convex when its region is a convex set, and the plainest way to say that is: it holds every halfsquare whose centroid lies inside the hull of its corners, and nothing else. Six times the centroid is an integer, and so are the corners doubled, so the test is exact.

⟨ Functions 4 ⟩ +≡

```

func (h hs) cent6()(int,int) {
  switch h.q {
  case 0:
    return 6 * h.x + 3, 6 * h.y + 1
  case 1:
    return 6 * h.x + 5, 6 * h.y + 3
  case 2:
    return 6 * h.x + 3, 6 * h.y + 5
  }
  return 6 * h.x + 1, 6 * h.y + 3
}

func (h hs) verts() [3][2]int {
  x, y := 2 * h.x, 2 * h.y
  c := [2]int{x + 1, y + 1}
  switch h.q {
  case 0:
    return [3][2]int{{x, y}, {x + 2, y}, c}
  case 1:
    return [3][2]int">{{x + 2, y}, {x + 2, y + 2}, c}
  case 2:
    return [3][2]int">{{x + 2, y + 2}, {x, y + 2}, c}
  }
  return [3][2]int">{{x, y + 2}, {x, y}, c}
}

```

16. No centroid can land on the hull itself: the four edge directions of a convex polyabolo are the horizontal, the vertical and the two diagonals, and six times a centroid is never a multiple of six in the coordinate that would be needed. A hull edge of any other slope means the hull is not a union of halfsquares, and the shape is not convex either, so answering “outside” there is right too.

⟨ Functions 4 ⟩ +≡

```

func (s shape) convex() bool {
  var pts [[2]int
  for _, h := range s {
    for _, v := range h.verts() {
      pts = append(pts, v)
    }
  }
  hl := hull(pts)
  if len(hl) < 3 {
    return false
  }
  ⟨ Compare the shape with its hull 17 ⟩
}

```

17. $\langle \text{Compare the shape with its hull 17} \rangle \equiv$

```

in := map[hs]bool{ }
x0, y0, x1, y1 := s[0].x, s[0].y, s[0].x, s[0].y
for _, h := range s {
    in[h] = true
     $\langle \text{Widen the bounding box 18} \rangle$ 
}
n := 0
for y := y0; y ≤ y1; y++ {
    for x := x0; x ≤ x1; x++ {
        for q := 0; q < 4; q++ {
            h := hs{x, y, q}
            if ¬inside(hl, h) {
                continue
            }
            if ¬in[h] {
                return false
            }
            n++
        }
    }
}
return n ≡ len(s)

```

This code is used in section 16.

18. $\langle \text{Widen the bounding box 18} \rangle \equiv$

```

if h.x < x0 {
    x0 = h.x
}
if h.x > x1 {
    x1 = h.x
}
if h.y < y0 {
    y0 = h.y
}
if h.y > y1 {
    y1 = h.y
}

```

This code is used in section 17.

19. The hull corners are doubled and the centroid is sixfold, so tripling the corners puts them in the same units.

⟨ Functions 4 ⟩ +≡

```

func inside(hl [][2]int, h hs) bool {
  cx, cy := h.cent6()
  for i := range hl {
    p, r := hl[i], hl[(i + 1) % len(hl)]
    if (3 * r[0] - 3 * p[0]) * (cy - 3 * p[1]) - (3 * r[1] - 3 * p[1]) * (cx - 3 * p[0]) ≤ 0 {
      return false
    }
  }
  return true
}

```

20. The hull itself is Andrew's monotone chain, which needs the points sorted and duplicates gone.

⟨ Functions 4 ⟩ +≡

```

func hull(pts [][2]int) [][2]int {
  ⟨ Sort the points and drop repeats 21 ⟩
  if len(pts) < 3 {
    return pts
  }
  var h [][2]int
  lower := 2
  for _, p := range pts {
    ⟨ Extend the chain by p 22 ⟩
  }
  lower = len(h) + 1
  for i := len(pts) - 2; i ≥ 0; i -- {
    p := pts[i]
    ⟨ Extend the chain by p 22 ⟩
  }
  return h[:len(h) - 1]
}

func cross(o, a, b [2]int) int {
  return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])
}

```

21. $\langle \text{Sort the points and drop repeats 21} \rangle \equiv$

```

sort.Slice(pts, func(i, j int) bool {
    if pts[i][0] != pts[j][0] {
        return pts[i][0] < pts[j][0]
    }
    return pts[i][1] < pts[j][1]
})
var uniq [][]int
for i, p := range pts {
    if i == 0 ∨ p != pts[i-1] {
        uniq = append(uniq, p)
    }
}
pts = uniq

```

This code is used in section 20.

22. The chain may be popped back to *lower* points and no further, which is 2 while the upper hull is being built and one more than the whole upper hull afterwards, so that the lower hull cannot eat it.

$\langle \text{Extend the chain by } p \text{ 22} \rangle \equiv$

```

for len(h) ≥ lower ∧ cross(h[len(h)-2], h[len(h)-1], p) ≤ 0 {
    h = h[:len(h)-1]
}
h = append(h, p)

```

This code is used in section 20.

23. The six parameters. Now the characterization itself. Answer 320 says that every convex polyabolo is an $m \times n$ rectangle— n wide and m tall— with right triangles of legs a, b, c, d cut from the lower left, lower right, upper right and upper left corners, subject to

$$a + b \leq n, \quad b + c \leq m, \quad c + d \leq n, \quad d + a \leq m,$$

and that it then has $N = 2mn - a^2 - b^2 - c^2 - d^2$ halvesquares.

⟨Types 3⟩ +≡

```
type param struct { m, n, a, b, c, d int }
```

24. ⟨Functions 4⟩ +≡

```
func (p param) size() int {
    return 2 * p.m * p.n - p.a * p.a - p.b * p.b - p.c * p.c - p.d * p.d
}
func (p param) legal() bool {
    return p.a + p.b ≤ p.n ∧ p.b + p.c ≤ p.m ∧ p.c + p.d ≤ p.n ∧ p.d + p.a ≤ p.m
}
```

25. A quarter belongs to the shape when its centroid survives all four cuts.

⟨Functions 4⟩ +≡

```
func (p param) region() shape {
    var s shape
    for y := 0; y < p.m; y++ {
        for x := 0; x < p.n; x++ {
            for q := 0; q < 4; q++ {
                ⟨Keep the quarter unless a corner cut takes it 26⟩
            }
        }
    }
    return s
}
```

26. ⟨Keep the quarter unless a corner cut takes it 26⟩ ≡

```
h := hs{x, y, q}
cx, cy := h.cent6()
switch {
case cx + cy < 6 * p.a:
case cx - cy > 6 * (p.n - p.b):
case cx + cy > 6 * (p.n + p.m - p.c):
case cy - cx > 6 * (p.m - p.d):
default:
    s = append(s, h)
}
```

This code is used in section 25.

27. The same shape arises from several parameter tuples, since the rectangle has symmetries of its own. Answer 320 kills the duplicates by asking for $m \leq n$ and for (a, b, c, d) to be lexicographically largest among the tuples the rectangle's symmetries produce: three rivals in general, seven when the rectangle is a square.

⟨ Functions 4 ⟩ +≡

```

func (p param) canonical() bool {
  if p.m > p.n {
    return false
  }
  t := [4]int{p.a, p.b, p.c, p.d}
  rivals := [][4]int{
    {p.b, p.a, p.d, p.c}, {p.c, p.d, p.a, p.b}, {p.d, p.c, p.b, p.a},
  }
  if p.m ≡ p.n {
    ⟨ Add the rivals of a square 28 ⟩
  }
  for _, r := range rivals {
    if ¬lexGE(t, r) {
      return false
    }
  }
  return true
}

func lexGE(u, v [4]int) bool {
  for i := 0; i < 4; i++ {
    if u[i] ≠ v[i] {
      return u[i] > v[i]
    }
  }
  return true
}

```

28. ⟨ Add the rivals of a square 28 ⟩ ≡

```

rivals = append(rivals,
  [4]int{p.a, p.d, p.c, p.b}, [4]int{p.b, p.c, p.d, p.a},
  [4]int{p.c, p.b, p.a, p.d}, [4]int{p.d, p.a, p.b, p.c})

```

This code is used in section 27.

29. The search needs a bound, and the answer supplies one: the smallest positive size with $m < n$ is $2m(n - m)$, and with $m = n$ it is $2n - 1$, so $n \leq (N + 2)/2$. Each cut obeys $a \leq \min(m, n) = m$ as well, and the loop is finite. The two minima are worth checking, since the bound rests on them; *smallest* finds them by trying every set of cuts.

⟨ Functions 4 ⟩ +≡

```

func smallest(m, n int) int {
  best := -1
  for a := 0; a ≤ m; a++ {
    for b := 0; b ≤ m; b++ {
      for c := 0; c ≤ m; c++ {
        for d := 0; d ≤ m; d++ {
          q := param{m, n, a, b, c, d}
          if q.legal() ∧ q.size() > 0 ∧ (best < 0 ∨ q.size() < best) {
            best = q.size()
          }
        }
      }
    }
  }
  return best
}

```

30.

⟨ Functions 4 ⟩ +≡

```

func convexAll(N int) []param {
  var out []param
  for m := 1; 2 * m ≤ N + 2; m++ {
    for n := m; 2 * n ≤ N + 2; n++ {
      if 2 * m * n < N {
        continue
      }
      ⟨ Try every set of corner cuts 31 ⟩
    }
  }
  ⟨ Sort the parameters 32 ⟩
  return out
}

```

31. $\langle \text{Try every set of corner cuts 31} \rangle \equiv$

```

for  $a := 0; a \leq m; a++$  {
  for  $b := 0; b \leq m; b++$  {
    for  $c := 0; c \leq m; c++$  {
      for  $d := 0; d \leq m; d++$  {
         $p := \text{param}\{m, n, a, b, c, d\}$ 
        if  $p.size() \equiv N \wedge p.legal() \wedge p.canonical()$  {
           $out = \text{append}(out, p)$ 
        }
      }
    }
  }
}

```

This code is used in section 30.

32. $\langle \text{Sort the parameters 32} \rangle \equiv$

```

 $sort.Slice(out, \text{func}(i, j \text{ int}) \text{ bool} \{$ 
   $u, v := out[i], out[j]$ 
   $a := [6]\text{int}\{u.m, u.n, u.a, u.b, u.c, u.d\}$ 
   $b := [6]\text{int}\{v.m, v.n, v.a, v.b, v.c, v.d\}$ 
  for  $k := 0; k < 6; k++$  {
    if  $a[k] \neq b[k]$  {
      return  $a[k] < b[k]$ 
    }
  }
  return false
 $\})$ 

```

This code is used in section 30.

33. Printing a shape's name the way the answer does.

$\langle \text{Functions 4} \rangle + \equiv$

```

func ( $p \text{ param}$ )  $String()$  string {
  return  $fmt.Sprintf("(\%d_\times_\%d;_\%d,\%d,\%d,\%d)", p.m, p.n, p.a, p.b, p.c, p.d)$ 
}

```


34. The fourteen tetraboloes. Exercise 319 draws the fourteen tetraboloes and names them A to N. Here they are, each as its four halvesquares; the letters are read off that figure, and the program checks below that this is exactly the set its own growth finds.

⟨ Functions 4 ⟩ +≡

```

func tetraboloes() map[string]piece {
  return map[string]piece{
    "A": {{0, 0, 1}, {0, 0, 3}, {0, 1, 1}, {1, 0, 3}},
    "B": {{0, 0, 0}, {0, 0, 2}, {0, 1, 0}, {0, 1, 2}},
    "C": {{0, 0, 1}, {0, 1, 2}, {1, 0, 3}, {1, 1, 0}},
    "D": {{0, 0, 2}, {0, 1, 1}, {1, 0, 3}, {1, 1, 0}},
    "E": {{0, 0, 1}, {1, 0, 1}, {1, 0, 3}, {2, 0, 0}},
    "F": {{0, 0, 2}, {0, 1, 1}, {1, 1, 3}, {1, 2, 0}},
    "G": {{0, 0, 1}, {0, 0, 3}, {0, 1, 1}, {1, 1, 0}},
    "H": {{0, 0, 1}, {0, 0, 3}, {0, 1, 1}, {1, 1, 3}},
    "I": {{0, 0, 1}, {1, 0, 3}, {1, 1, 1}, {2, 1, 3}},
    "J": {{0, 0, 1}, {1, 0, 3}, {1, 1, 1}, {2, 1, 0}},
    "K": {{0, 0, 1}, {0, 1, 1}, {1, 0, 3}, {1, 1, 0}},
    "L": {{0, 0, 0}, {0, 0, 2}, {0, 1, 1}, {1, 0, 0}},
    "M": {{0, 0, 0}, {0, 0, 2}, {0, 1, 0}, {1, 0, 0}},
    "N": {{0, 0, 1}, {1, 0, 1}, {1, 0, 3}, {2, 0, 3}},
  }
}

func letters() []string {
  return []string{"A", "B", "C", "D", "E", "F", "G",
    "H", "I", "J", "K", "L", "M", "N"}
}

```

35. “Exactly five of the tetraboloes,” says answer 320, “namely $\{E, G, J, K, L\}$, have an odd number of unmatched $\sqrt{2}$ sides in each direction.” A halvesquare’s hypotenuse is the diagonal of its cell, and it is unmatched when the other half of that cell is not in the piece.

⟨ Functions 4 ⟩ +≡

```

func (p piece) unmatched()(int, int) {
  in := map[half]bool{ }
  for _, h := range p {
    in[h] = true
  }
  var back, fwd int
  for _, h := range p {
    if in[half{h.x, h.y, (h.t + 2) % 4}] {
      continue
    }
    if h.t % 2 == 0 {
      back++
    } else {
      fwd++
    }
  }
  return back, fwd
}

```

36. Why “in each direction” is one condition and not two: a quarter turn sends halves of one kind to the other, so the two counts trade places, and their sum is 4. Hence they have the same parity, and the property belongs to the free piece and not to the way it is turned.

The consequence is the one the answer draws. Inside a packing the $\sqrt{2}$ edges match in pairs, so the number of unmatched “” sides, summed over the fourteen pieces, has the parity of the number of “” sides on the boundary of the region. Five pieces are odd, so that boundary count must be odd. The “” part of a convex region’s boundary is the lower left cut and the upper right one, of lengths a and c ; the “/” part is b and d . So $a + c$ and $b + d$ must both be odd.

⟨ Functions 4 ⟩ +≡

```
func (p param) parityOK() bool {
    return (p.a + p.c) % 2 ≡ 1 ∧ (p.b + p.d) % 2 ≡ 1
}
```

37. Packing. The exact cover problem has one item per quarter of the region and one per piece, and one option per placement. Quarters rather than halvesquares, again because a cell can be cut two ways; on quarters a placement is just a set of eight items.

⟨ Functions 4 ⟩ +≡

```

func packInput(p param, ps map[string]piece)(string, int) {
    reg := map[hs]bool{ }
    for _, h := range p.region() {
        reg[h] = true
    }
    var b strings.Builder
    ⟨ Write the items 38 ⟩
    n := 0
    for _, name := range letters() {
        for _, o := range ps[name].orients() {
            ⟨ Write every placement of o 39 ⟩
        }
    }
    return b.String(), n
}

func qname(h hs) string { return fmt.Sprintf("c%02d.%02d.%d", h.x, h.y, h.q) }

```

38. ⟨ Write the items 38 ⟩ ≡

```

var items []string
for h := range reg {
    items = append(items, qname(h))
}
sort.Strings(items)
for _, name := range letters() {
    fmt.Fprintf(&b, "%s□", name)
}
b.WriteString(strings.Join(items, "□"))
b.WriteString("\\n")

```

This code is used in section 37.

39. $\langle$ Write every placement of o 39 $\rangle \equiv$

```

q := o.quarters().norm()
for dy := 0; dy < p.m; dy++ {
  for dx := 0; dx < p.n; dx++ {
    var cells []string
    for _, c := range q {
      g := hs{c.x + dx, c.y + dy, c.q}
      if ¬reg[g] {
        cells =  $\Lambda$ 
        break
      }
      cells = append(cells, qname(g))
    }
    if cells  $\equiv$   $\Lambda$  {
      continue
    }
    sort.Strings(cells)
    fmt.Fprintf(&b, "%s□%s\n", name, strings.Join(cells, "□"))
    n++
  }
}

```

This code is used in section 37.

40. Two of the eight counts are printed by the answer as $2 \cdot 236$ and $2 \cdot 747$, and the reason is that those two regions are symmetric: every packing has a mirror image that is another packing. This counts the symmetries a region has, so that the doubling can be seen rather than assumed.

$\langle$ Functions 4 $\rangle + \equiv$

```

func (p param) selfSyms() int {
  reg := p.region().norm()
  want := reg.key()
  n := 0
  for k := 0; k < 8; k++ {
    t := make(shape, len(reg))
    for i, h := range reg {
      t[i] = transform(h, k)
    }
    if t.norm().key()  $\equiv$  want {
      n++
    }
  }
  return n
}

```

41. Doing what the mode asks. Everything is in place; what is left is to run it and print.

⟨Do what the mode asks 41⟩ ≡

```

if *mode ≡ "census" ∨ *mode ≡ "all" {
  ⟨Take the census 42⟩
}
if *mode ≡ "pieces" ∨ *mode ≡ "all" {
  ⟨Check the fourteen tetraboloes 43⟩
}
if *mode ≡ "bound" ∨ *mode ≡ "all" {
  ⟨Check the smallest sizes 45⟩
}
if *mode ≡ "convex" ∨ *mode ≡ "all" {
  ⟨List the convex shapes 46⟩
}
if *mode ≡ "pack" ∨ *mode ≡ "all" {
  ⟨Pack what is left 47⟩
}

```

This code is used in section 1.

42. The census is the expensive part—the number of polyaboloes grows by a factor of about 3.6 each time—so it stops at 11 unless told otherwise. Every line has to agree, and that is the evidence that the six parameters miss nothing and count nothing twice.

⟨Take the census 42⟩ ≡

```

fmt.Println("Npolyaboloesconvexbyparameters")
all := growH(*upto)
for n := 1; n ≤ *upto; n++ {
  cv := 0
  for _, p := range all[n] {
    if p.quarters().norm().convex() {
      cv++
    }
  }
  mark := "ok"
  if cv ≠ len(convexAll(n)) {
    mark = "MISMATCH"
  }
  fmt.Printf("%2d_%11d_%8d_%14d_%s\n", n, len(all[n]), cv,
    len(convexAll(n)), mark)
}

```

This code is used in section 41.

43. The tetraboloos are checked against the growth, and then O’Beirne’s five are named.

⟨ Check the fourteen tetraboloos 43 ⟩ ≡

```

ps := tetraboloos()
⟨ Match the lettered pieces with the grown ones 44 ⟩
var odd []string
for _, name := range letters() {
    back, fwd := ps[name].unmatched()
    if back % 2 ≠ fwd % 2 {
        fmt.Printf("piece_%s:_parities_differ,_%d_and_%d\n", name, back, fwd)
    }
    if back % 2 ≡ 1 {
        odd = append(odd, name)
    }
}
fmt.Printf("odd_in_each_direction:%v\n", odd)

```

This code is used in section 41.

44. ⟨ Match the lettered pieces with the grown ones 44 ⟩ ≡

```

mine := map[string]string{}
for _, p := range growH(4)[4] {
    mine[p.canon()] = ""
}
for _, name := range letters() {
    k := ps[name].canon()
    if _, ok := mine[k]; ¬ok {
        fmt.Printf("piece_%s_is_not_a_tetrabolo_I_grew\n", name)
    }
    mine[k] = name
}
for k, v := range mine {
    if v ≡ "" {
        fmt.Printf("tetrabolo_%s_got_no_letter\n", k)
    }
}
fmt.Printf("%d_tetraboloos,_all_fourteen_lettered\n", len(mine))

```

This code is used in section 43.

45. The claim that bounds the search, over a range wide enough to be convincing.

⟨Check the smallest sizes 45⟩ ≡

```

bad := 0
for m := 1; m ≤ 12; m++ {
  for n := m; n ≤ 24; n++ {
    want := 2 * m * (n - m)
    if m ≡ n {
      want = 2 * n - 1
    }
    if smallest(m, n) ≠ want {
      bad++
      fmt.Printf("m=%d n=%d: smallest positive size %d, not %d\n",
        m, n, smallest(m, n), want)
    }
  }
}
fmt.Printf("smallest positive sizes: %d disagreements\n", bad)

```

This code is used in section 41.

46. Listing the convex N -aboloes, with the parity test applied. Each one is also put through the convexity test of the previous part, so that the parameters and the geometry agree shape by shape.

⟨List the convex shapes 46⟩ ≡

```

cs := convexAll(*size)
pass := 0
for _, p := range cs {
  r := p.region().norm()
  if ¬r.convex() ∨ len(r) ≠ 2 * p.size() {
    fmt.Printf("%v is not a convex %d-abolo\n", p, *size)
  }
  if p.parityOK() {
    pass++
    fmt.Printf("%v passes the parity test\n", p)
  }
}
fmt.Printf("%d convex %d-aboloes, %d pass\n", len(cs), *size, pass)

```

This code is used in section 41.

47. And at last the packings.

⟨Pack what is left 47⟩ ≡

```

for _, p := range convexAll(*size) {
  if ¬p.parityOK() {
    continue
  }
  ⟨Count the packings of p 48⟩
}

```

This code is used in section 41.

48. $\langle$ Count the packings of p 48 $\rangle \equiv$

```

in, nopt := packInput(p, tetraboloes())
t0 := time.Now()
xc := cells.NewXCC()
k := 0
for range xc.Dance(strings.NewReader(in)).Solutions {
    k++
}
fmt.Printf("%v%doptions,%7d%solutions,%2d%symmetries,%10dnodes,%s\n",
    p, nopt, k, p.selfSyms(), xc.Nodes(), time.Since(t0).Round(time.Millisecond))

```

This code is used in section 47.

49. Index.

- a*: [20](#), [23](#), [24](#), [26](#), [27](#), [28](#), [29](#), [31](#), [32](#), [33](#), [36](#).
all: [11](#), [12](#), [42](#).
append: [8](#), [10](#), [12](#), [13](#), [16](#), [21](#), [22](#), [26](#), [28](#), [31](#), [38](#), [39](#), [43](#).
b: [8](#), [20](#), [23](#), [24](#), [26](#), [27](#), [28](#), [29](#), [31](#), [32](#), [33](#), [36](#), [37](#), [38](#), [39](#).
back: [35](#), [43](#).
bad: [45](#).
best: [10](#), [29](#).
bool: [8](#), [10](#), [11](#), [12](#), [16](#), [17](#), [19](#), [21](#), [24](#), [27](#), [32](#), [35](#), [36](#), [37](#).
Builder: [8](#), [37](#).
c: [15](#), [23](#), [24](#), [26](#), [27](#), [28](#), [29](#), [31](#), [32](#), [33](#), [36](#), [39](#).
canon: [10](#), [12](#), [44](#).
canonical: [27](#), [31](#).
cells: [1](#), [39](#), [48](#).
census: [2](#).
cent6: [15](#), [19](#), [26](#).
convex: [2](#), [16](#), [42](#), [46](#).
convexAll: [30](#), [42](#), [46](#), [47](#).
cross: [20](#), [22](#).
cs: [46](#).
cv: [42](#).
cx: [19](#), [26](#).
cy: [19](#), [26](#).
d: [23](#), [24](#), [26](#), [27](#), [28](#), [29](#), [31](#), [32](#), [33](#), [36](#).
Dance: [48](#).
dx: [39](#).
dy: [39](#).
flag: [2](#).
fmt: [8](#), [33](#), [37](#), [38](#), [39](#), [42](#), [43](#), [44](#), [45](#), [46](#), [48](#).
Fprintf: [8](#), [38](#), [39](#).
fwd: [35](#), [43](#).
g: [12](#), [39](#).
growH: [11](#), [42](#), [44](#).
h: [4](#), [5](#), [6](#), [7](#), [8](#), [9](#), [10](#), [12](#), [13](#), [15](#), [16](#), [17](#), [18](#), [19](#), [20](#), [22](#), [26](#), [35](#), [37](#), [38](#), [40](#).
half: [3](#), [4](#), [5](#), [11](#), [13](#), [35](#).
hl: [16](#), [17](#), [19](#).
hs: [3](#), [4](#), [5](#), [6](#), [7](#), [12](#), [15](#), [17](#), [19](#), [26](#), [37](#), [39](#).
hull: [16](#), [20](#).
i: [5](#), [8](#), [9](#), [10](#), [19](#), [20](#), [21](#), [27](#), [32](#), [40](#).
in: [17](#), [35](#), [48](#).
inside: [17](#), [19](#).
Int: [2](#).
int: [3](#), [5](#), [8](#), [11](#), [13](#), [15](#), [16](#), [19](#), [20](#), [21](#), [23](#), [24](#), [27](#), [28](#), [29](#), [30](#), [32](#), [35](#), [37](#), [40](#).
items: [38](#).
j: [8](#), [21](#), [32](#).
Join: [38](#), [39](#).
k: [5](#), [10](#), [12](#), [32](#), [40](#), [44](#), [48](#).
key: [8](#), [10](#), [40](#).
legal: [24](#), [29](#), [31](#).
len: [10](#), [16](#), [17](#), [19](#), [20](#), [22](#), [40](#), [42](#), [44](#), [46](#).
letters: [34](#), [37](#), [38](#), [43](#), [44](#).
lexGE: [27](#).
lower: [20](#), [22](#).
m: [23](#), [24](#), [25](#), [26](#), [27](#), [29](#), [30](#), [31](#), [32](#), [33](#), [39](#), [45](#).
main: [1](#).
make: [10](#), [11](#), [40](#).
mark: [42](#).
Millisecond: [48](#).
mine: [44](#).
mode: [2](#), [41](#).
mx: [9](#).
my: [9](#).
N: [30](#), [31](#).
n: [11](#), [12](#), [17](#), [23](#), [24](#), [25](#), [26](#), [27](#), [29](#), [30](#), [31](#), [32](#), [33](#), [37](#), [39](#), [40](#), [42](#), [45](#).
name: [37](#), [38](#), [39](#), [43](#), [44](#).
neighbours: [12](#), [13](#).
NewReader: [48](#).
NewXCC: [48](#).
Nodes: [48](#).
nopt: [48](#).
norm: [8](#), [39](#), [40](#), [42](#), [46](#).
Now: [48](#).
o: [20](#), [37](#), [39](#).
odd: [43](#).
ok: [44](#).
orients: [10](#), [37](#).
out: [10](#), [13](#), [30](#), [31](#), [32](#).
p: [10](#), [11](#), [12](#), [19](#), [20](#), [21](#), [22](#), [24](#), [25](#), [26](#), [27](#), [28](#), [31](#), [33](#), [35](#), [36](#), [37](#), [39](#), [40](#), [42](#), [44](#), [46](#), [47](#), [48](#).
pack: [2](#).
packInput: [37](#), [48](#).
param: [23](#), [24](#), [25](#), [27](#), [29](#), [30](#), [31](#), [33](#), [36](#), [37](#), [40](#).
parityOK: [36](#), [46](#), [47](#).
Parse: [2](#).
pass: [46](#).
piece: [3](#), [10](#), [11](#), [12](#), [34](#), [35](#), [37](#).
pieces: [2](#).
Printf: [42](#), [43](#), [44](#), [45](#), [46](#), [48](#).
Println: [42](#).

ps: [37](#), [43](#), [44](#).
pts: [16](#), [20](#), [21](#).
q: [3](#), [6](#), [7](#), [8](#), [10](#), [12](#), [15](#), [17](#), [25](#), [26](#), [29](#), [37](#), [39](#).
qname: [37](#), [38](#), [39](#).
quarters: [4](#), [10](#), [12](#), [39](#), [42](#).
r: [19](#), [27](#), [46](#).
reg: [37](#), [38](#), [39](#), [40](#).
region: [25](#), [37](#), [40](#), [46](#).
rivals: [27](#), [28](#).
Round: [48](#).
s: [8](#), [10](#), [13](#), [14](#), [16](#), [17](#), [25](#), [26](#).
seen: [10](#), [11](#), [12](#).
selfSyms: [40](#), [48](#).
shape: [3](#), [8](#), [10](#), [16](#), [25](#), [40](#).
Since: [48](#).
size: [2](#), [24](#), [29](#), [31](#), [46](#), [47](#).
Slice: [8](#), [21](#), [32](#).
smallest: [29](#), [45](#).
Solutions: [48](#).
sort: [8](#), [21](#), [32](#), [38](#), [39](#).
Sprintf: [33](#), [37](#).
String: [2](#), [8](#), [33](#), [37](#).
string: [8](#), [10](#), [11](#), [33](#), [34](#), [37](#), [38](#), [39](#), [43](#), [44](#).
Strings: [38](#), [39](#).
strings: [8](#), [37](#), [38](#), [39](#), [48](#).
t: [3](#), [4](#), [5](#), [8](#), [9](#), [10](#), [12](#), [13](#), [27](#), [35](#), [40](#).
t0: [48](#).
tetraboloos: [34](#), [43](#), [48](#).
time: [48](#).
transform: [5](#), [40](#).
transformH: [5](#), [10](#).
u: [13](#), [27](#), [32](#).
uniq: [21](#).
unmatched: [35](#), [43](#).
upto: [2](#), [11](#), [42](#).
used: [12](#).
v: [16](#), [27](#), [32](#), [44](#).
verts: [15](#), [16](#).
want: [40](#), [45](#).
WriteString: [38](#).
x: [3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [9](#), [13](#), [14](#), [15](#), [17](#), [18](#), [25](#), [26](#), [35](#),
[37](#), [39](#).
x0: [17](#), [18](#).
x1: [17](#), [18](#).
xc: [48](#).
y: [3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [9](#), [13](#), [14](#), [15](#), [17](#), [18](#), [25](#), [26](#), [35](#),
[37](#), [39](#).
y0: [17](#), [18](#).

- ⟨Add one triangle to p in every way 12⟩ Used in section 11
- ⟨Add the rivals of a square 28⟩ Used in section 27
- ⟨Check the fourteen tetraboloes 43⟩ Used in section 41
- ⟨Check the smallest sizes 45⟩ Used in section 41
- ⟨Compare the shape with its hull 17⟩ Used in section 16
- ⟨Count the packings of p 48⟩ Used in section 47
- ⟨Do what the mode asks 41⟩ Used in section 1
- ⟨Extend the chain by p 22⟩ Used in section 20
- ⟨Functions 4⟩ Used in section 1
- ⟨Keep the quarter unless a corner cut takes it 26⟩ Used in section 25
- ⟨List the convex shapes 46⟩ Used in section 41
- ⟨Match the lettered pieces with the grown ones 44⟩ Used in section 43
- ⟨Pack what is left 47⟩ Used in section 41
- ⟨Read the command line 2⟩ Used in section 1
- ⟨Reflect the quarter 6⟩ Used in section 5
- ⟨Slide the shape to the origin 9⟩ Used in section 8
- ⟨Sort the parameters 32⟩ Used in section 30
- ⟨Sort the points and drop repeats 21⟩ Used in section 20
- ⟨Step across side s 14⟩ Used in section 13
- ⟨Take the census 42⟩ Used in section 41
- ⟨Try every set of corner cuts 31⟩ Used in section 30
- ⟨Turn the quarter 7⟩ Used in section 5
- ⟨Types 3⟩ Used in section 1
- ⟨Widen the bounding box 18⟩ Used in section 17
- ⟨Write every placement of o 39⟩ Used in section 37
- ⟨Write the items 38⟩ Used in section 37

Convex Polyaboloes

	Section	Page
Introduction	1	1
The grid	3	3
Growing polyaboloes	11	7
Convexity	15	9
The six parameters	23	13
The fourteen tetraboloes	34	17
Packing	37	19
Doing what the mode asks	41	21
Index	49	25